

Stage 1 — Building the Data Layer

Technical Report on Python Architecture, Implementation & Benchmarking Analysis

Course:	Big Data — B.Sc. Data Science and Engineering (ULPGC)	Date:	October 2026
Team:	TheScratchers	Branch:	ft/python-setup (Python 3.9+)
Author:	Aimar Daniel Alejandro Santana	Repository:	github.com/TheScratchers/stage_1

1. Executive Summary & Specification Compliance

This report presents the architecture, implementation details, and empirical benchmark results of the **Data Layer (Stage 1)** developed in Python for the Project Gutenberg search engine. The implementation follows the cross-language benchmark contract (*shared/CONTRACT.md*) agreed with the Java and C++ teammates, so that the numbers reported here are directly comparable with theirs: the same 20 real Gutenberg books, the same tokenizer rule, the same 10-word query workload, and the same synthetic scales.

Requirement (Contract)	Python Component	Implementation & Coverage
Shared 20-book dataset	<code>download_shared_dataset.py</code> , <code>shared_contract.py</code>	Downloads exactly the 20 contract books (<code>./shared/books.txt</code>) into <code>datalake_shared/</code> ; idempotent, re-run-safe. All three benchmarks replicate this real content across synthetic ids rather than inventing their own corpus.
Ingestion & Header/Body Split	<code>download_book.py</code>	Fetches a book from Project Gutenberg and splits it into <code><id>.header.txt</code> / <code><id>.body.txt</code> , consumed by every datalake layout and by <code>metadata.py</code> .
3 Datalake Layouts	<code>datalake.py</code> , <code>datalake_book.py</code> , <code>datalake_batch.py</code>	Time-based (<code>YYYYMMDD/HH/</code>), book-based (<code><id>/</code>), and batch-based (<code>batch_N-M/</code> , 1,000-id ranges) — three interchangeable path-resolution strategies sharing the same ingestion/benchmark harness.
Tokenizer / normalization rule	<code>inverted_index*.py</code> (<code>TOKEN_PATTERN</code>)	Lowercases first, then extracts maximal runs of <code>[A-Za-z]+</code> ; no stemming, no stopwords removal — exactly the rule mandated in <i>CONTRACT.md</i> Section 2.
3 Inverted Indexes	<code>inverted_index.py</code> , <code>inverted_index_hierarchical.py</code> , <code>inverted_index_mongo.py</code>	Monolithic JSON file, hierarchical per-letter/per-term files, and a real MongoDB collection — the three structures the team agreed on, not a substitute format.
Standard Query Workload	<code>shared_contract.py</code> (<code>load_words()</code>)	Parses the 10 fixed terms from <code>./shared/words.txt</code> (4 common / 4 medium / 2 rare), used identically for query and update benchmarks.

Metadata Datamart (SQLite)	<code>metadata.py</code>	Schema with <code>book_id</code> primary key, title/author/language, plus body/header paths; <code>upsert_book</code> , <code>find_by_id</code> , <code>find_by_author</code> exercised directly (no idealized bulk-insert bypass).
Metadata Benchmark (contract extension)	<code>benchmark_metadata.py</code>	Insertion speed, <code>find_by_id</code> , and <code>find_by_author</code> (Dickens/Carroll, timed separately) at the 4 contract scales (100 / 1,000 / 10,000 / 100,000).
Control Layer & State Tracking	<code>control.py</code>	Orchestrates one download or one indexing step at a time, tracking progress in <code>control/downloaded_books.txt</code> and <code>indexed_books.txt</code> to avoid duplicate work.
Datalake / Index / Metadata reporting (Section 6)	<code>benchmark_datalake.py</code> , <code>benchmark_inverted_index.py</code> , <code>benchmark_metadata.py</code>	Every metric Section 6 lists — write throughput, lookup cost, incremental-processing cost, recovery behavior and storage overhead for the datalake; indexing speed, query performance, update performance, memory/disk usage for the index — is reported at every one of the 3 scales , not as a single flat snapshot.

This last row is deliberate: an earlier draft of the C++ report in this same repository (*c++/stage_1_cpp_report.pdf*) reports its datalake-layout benchmark as a single row per structure with no scale variation, and reports its third inverted-index structure as a custom binary format rather than the MongoDB collection the team agreed on. Both are addressed here by construction: Sections 4.1 and 4.2 below show the full 3-scale progression for every structure, and MongoDB is exercised as a live database, not substituted.

2. System Architecture & Design Principles

The codebase separates raw storage (**Datalake**), query-optimized derived stores (**Datamarts**), and pipeline orchestration (**Control Layer**), with a thin **shared-contract** layer gluing the benchmark scripts to the cross-language agreement.

- **Datalake Layer:** the repository for ingested book texts. Exposes three interchangeable partitioning layouts (time-based, book-based, batch-based), each a small module providing a single `*_path()` function; all three are driven by the same `download_book()` call and evaluated by the same benchmark harness, so the only thing that differs between them is where a book's two files end up on disk.
- **Metadata Datamart:** extracts title, author and language from each book's header via regular expressions and stores them in SQLite, keyed by `book_id`, alongside pointers back to the body/header files in the datalake.
- **Inverted Index Datamart:** tokenizes body texts under the shared contract's rule and maintains postings (`term` → sorted `book_ids`) across three physical representations with very different cost profiles: a single JSON file (cheap once loaded, expensive to touch at scale), a folder-per-letter/file-per-term layout (no load step, but one filesystem call per lookup), and a MongoDB collection with a unique index on `term` (a real query engine, at the cost of a running database process).
- **Control Layer:** a minimal orchestrator that advances the pipeline one step at a time (download a new book, or index one already-downloaded one — never both in the same step), persisting progress in two plain-text files so a run can be stopped and resumed without re-processing anything.
- **Shared-contract layer:** `shared_contract.py` is the single place that parses `../shared/CONTRACT.md`'s companion files (`books.txt`, `words.txt`) and the two fixed metadata-benchmark authors, so the exact same

20 books and 10 query words are used identically by all three benchmark scripts instead of being copy-pasted three times.

3. Modular Codebase Breakdown

Each module has one responsibility: one file per datalake layout, one file per index structure, implementation code kept separate from its benchmark harness. Total: **2,228 LOC** across 14 source files, plus a **594-LOC** pytest suite (6 files, 24 tests) covering every module except `inverted_index_mongo.py`, which needs a live database and is instead exercised manually via the `--mongo` benchmark flag.

File	LOC	Responsibility
<code>shared_contract.py</code>	47	Parses the shared contract's <code>books.txt/words.txt</code> and the 2 fixed metadata-query authors — the single source of truth all 3 benchmark scripts read from.
<code>datalake_book.py</code>	55	Book-based datalake layout: one dedicated directory per book.
<code>datalake_batch.py</code>	71	Batch-based datalake layout: books grouped into fixed-width id-range folders (default 1,000 ids/folder).
<code>datalake.py</code>	73	Time-based datalake layout (YYYYMMDD/HH/) plus the shared <code>ingest_book()</code> entry point.
<code>download_book.py</code>	104	Fetches one book from Project Gutenberg and splits it into header/body text files.
<code>download_shared_dataset.py</code>	105	Idempotently downloads the 20 contract books into <code>datalake_shared/</code> .
<code>inverted_index.py</code>	128	Monolithic single-JSON-file inverted index: build, search, incremental update.
<code>inverted_index_hierarchical.py</code>	141	Folder-per-letter/file-per-term inverted index, with fine-grained per-term update.
<code>inverted_index_mongo.py</code>	142	MongoDB-backed inverted index, with a unique index on term and <code>\$addToSet</code> -based update.
<code>metadata.py</code>	174	SQLite metadata datamart: schema, header parsing, <code>upsert/find_by_id/find_by_author</code> .
<code>control.py</code>	178	Pipeline orchestrator; tracks downloaded/indexed state across two control files.
<code>benchmark_metadata.py</code>	213	Metadata benchmark harness: insertion speed, query performance, scalability at 4 scales.
<code>benchmark_datalake.py</code>	315	Datalake benchmark harness: write throughput, lookup, incremental processing, recovery, storage overhead, 3 structures × 3 scales.
<code>benchmark_inverted_index.py</code>	482	Inverted-index benchmark harness: build time, 10-word query workload, update performance, peak memory, storage overhead, 3 structures × 3 scales.

4. Experimental Results & Benchmark Analysis

All benchmarks were executed on the author's Windows 11 machine, against the 20 real Project Gutenberg contract books (shared/books.txt, fetched via download_shared_dataset.py), replicated across synthetic book_ids at the 3 contract scales (100 / 1,000 / 10,000 books). Every run writes its disposable synthetic files to a scratch folder outside the project tree, so results reflect genuine I/O cost rather than cloud-sync interference.

4.1 Datalake Layout Benchmark — full scale progression

Unlike a single-point snapshot, every metric below is reported at all 3 contract scales for all 3 layouts, so the actual scalability trend — not just one data point — is visible.

Layout	Scale	Write (books/s)	Lookup avg (ms)	Dirs created	Max files/dir
time_based (YYYYMMDD/HH/)	100	207.0	14.35	2	100
book_based (<id>/)	100	225.3	13.44	100	1
batch_based (batch_N/)	100	190.7	13.13	1	100
time_based	1,000	225.3	12.86	2	1,000
book_based	1,000	199.4	13.57	1,000	1
batch_based	1,000	208.4	13.44	1	1,000
time_based	10,000	201.8	15.23	2	10,000
book_based	10,000	201.4	13.41	10,000	1
batch_based	10,000	233.6	12.54	10	1,000

Layout	Scale	Incr. detect (s)	Incr. write (s)	Recovery resumed	Recovery (s)	Recovery OK
time_based	100	0.0028	0.0342	50	0.233	✓
book_based	100	0.0020	0.0381	50	0.260	✓
batch_based	100	0.0019	0.0343	50	0.198	✓
time_based	1,000	0.0266	0.454	500	2.426	✓
book_based	1,000	0.0282	0.455	500	2.427	✓
batch_based	1,000	0.0298	0.568	500	2.528	✓
time_based	10,000	0.2756	4.199	5,000	22.141	✓
book_based	10,000	0.2797	4.861	5,000	24.782	✓
batch_based	10,000	0.1748	3.740	5,000	22.421	✓

Analysis:

- **Write throughput is close at small scale, diverges at 10,000:** at 100–1,000 books the three layouts stay within ± 10 –15% of each other (dominated by per-file I/O rather than directory structure), but at 10,000 books batch_based pulls ahead (233.6 books/s vs. ~ 201.4 –201.8 for the other two) — it caps every directory at 1,000 entries (growing the directory count instead: $1 \rightarrow 1 \rightarrow 10$ as scale increases tenfold each time), avoiding both time_based's single hot directory (2 dirs holding all 10,000 files at the top scale) and book_based's one-directory-per-book overhead (10,000 directory creations).

- **Storage/structural overhead is the direct trade-off the contract asks for:** `time_based` keeps directory count flat (2) regardless of volume, at the cost of an unbounded single directory; `book_based` keeps every directory at exactly 1 file, at the cost of one directory per book (10,000 at the top scale); `batch_based` trades a configurable middle ground (1,000 files/directory by default).
- **Recovery cost scales with how many existing entries must be re-verified, not just with total volume:** all three layouts correctly resumed every already-ingested book without re-writing it (`recovery_ok = true` at every scale), but `book_based` was consistently the slowest to recover at 10,000 books (24.78 s vs. 22.14–22.42 s for the other two), since resuming means statting 10,000 individual per-book directories instead of scanning a shared one.
- **Incremental-processing cost (detecting + writing new books without re-reading the existing set) stays a small fraction of a full rebuild at every scale** — at 10,000 books, detecting and writing ~1,000 new candidates takes 0.17–0.28 s + 3.7–4.9 s, versus 42.8–49.6 s for a full 10,000-book write — confirming all three layouts support cheap append-only ingestion on top of an already-populated datalake.

4.2 Inverted Index Benchmark — all 3 structures, true MongoDB included

The vocabulary size (**35,008 unique terms**) is identical across every scale and every structure: synthetic volume is generated by replicating the 20 real books' content across new `book_ids`, so the set of distinct words never grows — only each term's postings list does. That is exactly what should drive the cost differences below.

Structure	Scale	Build (s)	Peak memory (MB)	Disk footprint (MB)	Avg lookup (ms)	Update cost (s)
json_monolithic	100	14.96	67.97	5.81	0.0019	0.386
hierarchical	100	38.01	67.96	5.29	1.960	6.010
mongodb	100	21.77	82.33	n/a (external DB)	33.78	2.779
json_monolithic	1,000	149.81	412.32	53.83	0.0027	3.109
hierarchical	1,000	146.82	375.88	53.31	5.699	14.453
mongodb	1,000	137.20	472.09	n/a (external DB)	10.715	2.677
json_monolithic	10,000	1617.86	4615.03	534.07	0.0466	50.829
hierarchical	10,000	1547.12	4108.95	533.55	9.668	29.131
mongodb	10,000	1279.21	4685.46	n/a (external DB)	46.75	4.441

In-Depth Analysis & Trade-offs:

- **Peak memory is almost identical across structures at every scale** (4.11–4.69 GB at 10,000 books) because all three `build_index()` implementations do the exact same first pass — scanning every body file and accumulating a complete `term → book_ids` dictionary in memory — before persisting it differently; the storage structure only changes what happens to that dictionary *after* it is built, not how big it gets while being built.
- **JSON monolithic's query cost stays negligible (<0.05 ms even at 10,000 books)** once the one-time load has happened, since every lookup after that is a plain in-memory dict access — but its update cost grows almost linearly with the full file size (0.39 s at a 5.8 MB file → 50.83 s at a 534 MB file), because `update_index()` for this structure re-serializes and rewrites the *entire* file on every single change, making it unsuitable for frequent small updates at scale.
- **Hierarchical's update cost grows with scale despite only touching the handful of term files that appear in the new book** (6.01 s → 14.45 s → 29.13 s): its `update_index()` does a read-modify-write of each touched term file (load the existing postings, add the new id, rewrite the whole file) rather than a pure append,

so the cost tracks how large those specific postings lists have already grown at that scale, not the size of the rest of the index. Its per-query lookup is consistently more expensive than JSON's warm in-memory access (1.96–9.67 ms vs. 0.002–0.047 ms) because every single query pays a filesystem open+read with no equivalent "load once" step.

- **MongoDB is the only structure whose update cost stays close to flat across scale** (2.78 s → 2.68 s → 4.44 s) because each update is one indexed `$addToSet` against a single document, independent of the rest of the collection's size — but its measured per-query latency (10.7–46.8 ms, non-monotonic with scale) is dominated by connection overhead rather than document lookup cost: `search()` opens and closes a fresh `MongoClient` connection on every call, so the 10-word query benchmark mostly measures round-trip/handshake cost to the local database process, not index-lookup cost itself. This is a real, code-grounded characteristic of using an actual database as the third structure — one that a non-MongoDB substitute would never surface.

4.3 Metadata Storage Benchmark (SQLite stress test, contract Section 5)

Scale (books)	Insert (s)	Throughput (books/s)	find_by_id (ms)	Dickens lookup (ms)	Carroll lookup (ms)	DB size (KB)
100	0.694	144.1	0.525	0.505	0.507	24
1,000	4.969	201.2	0.399	0.827	0.818	132
10,000	51.119	195.6	0.650	8.00	5.28	1,224
100,000	525.891	190.2	0.572	57.38	34.64	12,164

Analysis:

- **Insertion throughput stays essentially flat (144–201 books/s) across 3 orders of magnitude:** `upsert_book()` opens, commits and closes its own SQLite connection on every single call — the actual production code path used by `control.py`, not an idealized bulk-insert bypass — so per-call connection overhead dominates at every scale and total insert time scales almost perfectly linearly with row count (0.69 s → 525.9 s, a ~758x increase for a 1,000x row-count increase).
- **find_by_id stays flat and sub-millisecond at every scale** (0.40–0.65 ms across a 1,000x table-size range), confirming SQLite's primary-key B-tree index makes this lookup's cost independent of table size.
- **find_by_author scales sharply with table size for both fixed authors**, since this query has no supporting index (a `LIKE` substring match performs a full table scan): the repeated author (*Charles Dickens*, 3 of the 20 real books, ~15% of synthetic rows) grows from 0.51 ms at 100 rows to 57.38 ms at 100,000 rows (~113x), while the unique author (*Lewis Carroll*, 1 of 20, ~5% of rows) grows from 0.51 ms to 34.64 ms (~68x) over the same range — confirming the contract's intended effect: at scale, the more frequent author's query costs noticeably more than the rarer one's (57.38 ms vs. 34.64 ms at 100,000 rows), a gap that barely exists yet at 100 rows.
- **Storage overhead per row shrinks as the table grows** (240 B/row at 100 books down to ~124 B/row at 100,000 books), reflecting SQLite's fixed page-level overhead being amortized across more rows.

5. Execution & Verification Guide

The Python implementation requires Python 3.9+ and the dependencies in `requirements.txt` (`requests`, `pymongo`); the MongoDB structure additionally needs a running MongoDB instance. All commands assume the current directory is `python/`.

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. One-time setup: download the 20 shared contract books
python src/download_shared_dataset.py

# 3. (Optional) start MongoDB for the 3rd index structure
docker run -d --name stage1-mongo -p 27017:27017 mongo:7

# 4. Single-book ingestion / indexing (manual smoke test)
python src/datalake.py 1342 5 11
python src/metadata.py
python src/inverted_index.py
python src/inverted_index_hierarchical.py
python src/control.py 3

# 5. Automated benchmarks (contract scales, default if omitted)
python src/benchmark_datalake.py 100,1000,10000
python src/benchmark_inverted_index.py 100,1000,10000
python src/benchmark_inverted_index.py --mongo 100,1000,10000
python src/benchmark_metadata.py 100,1000,10000,100000

# 6. Test suite (24 tests, no network/MongoDB required)
pip install pytest
python -m pytest tests/ -v
```

Results are written as JSON to `datamarts/benchmark_datalake_results.json`, `datamarts/benchmark_inverted_index_results.json` and `datamarts/benchmark_metadata_results.json` — the three files committed to this branch and the exact source data for every table in Section 4 above.